qsBye

折腾笔记[19]-使用rust发布和订阅ros2消息

摘要

使用rust发布和订阅ros2消息, 无需安装ros2环境.Publishing and subscribing to ROS2 messages using Rust without installing the ROS2 environment.

关键词

rust;ros2;DDS;

关键信息

项目地址:
[https://github.com/ByeIO/bye.rosette.rs/blob/rust/crates/rosette_core/examples/rosdds_pub_sub.rs]

```
[workspace.package]
name = "bye_rosette_rs"
version = "0.0.1"
authors = ["qsbye <2557877116@qq.com"]
repository = "https://github.com/ByeIO/bye.rosette.rs"
edition = "2024"
license = "Apache-2.0"

[workspace.dependencies]
# 错误处理
anyhow = "1.0.97"
# 时间格式化
chrono = "0.4.40"
# Model Context Protocol协议定义
rust-mcp-schema = { version = "0.2.2", path = "./static/rust-mcp-schema" }
# Model Context Protocol开发工具
rust-mcp-sdk = { version = "0.1.2", path = "./static/rust-mcp-sdk/crates/rust-mcp-sdk" }
# 多线程框架
tokio = { version = "1.44.1", features = ["full"] }
# ros2接口(针对humble)
ros2-interfaces-humble = { version = "0.0.1", path = "./static/ros2-interfaces-humble" }
# 日志后端
log = { version = "0.4.27", features = ["std"] }
# 日志前端
log4rs = "1.3.0"
# ros2数据包bag处理
rosbag2-rs = { version = "0.2.1", path = "./static/rosbag2-rs" }
# ros1数据包bag处理
rosbag = { version = "0.6.3", path = "./static/rosbag-rs" }
# 临时文件
tempfile = { version = "3.19.1" }
# ros2消息收发
ros2-client = { version = "0.8.1", features = ["pre-iron-gid"], path = "./static/ros2-client"
# DDS分布式通信
rustdds = { version = "0.11.4", path = "./static/RustDDS" }
# MQTT协议栈(纯Rust)
rumqttc = { version = "0.24.0", path = "./static/rumqttc" }
rumqttd = { version = "0.19.0", path = "./static/rumqttd" }
# 静态变量
static_cell = "2.1.0"
# 全局变量
once_cell = "1.21.3"
# 通用数据表达序列化/反序列化
cdr = { version = "0.2.4", path = "./static/cdr-rs" }
# 大小端字节序
byteorder = "1.5.0"
# 异步流处理
futures = "0.3.31"
```

原理简介

DDS通信技术简介:面向数据

[https://blog.csdn.net/qq_38880380/article/details/135131544]
[https://zhuanlan.zhihu.com/p/192981171]
[https://www.cnblogs.com/tomato-haha/p/17696510.html]

公告

昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0

‹ 2025年4月 ›

日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

搜索

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签

Rust(15)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类

51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(5)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(13)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(21)

随笔档案

2025年4月(2)
2025年3月(4)

[https://www.omg.org/spec/DDS/]
[https://www.omg.org/spec/DDSI-RTPS/2.2/]
DDS（Data Distribution Service），即数据分发服务，是OMG（Object Management Group）对象管理组织发布的分布式通信规范，采用订阅发布模型，以中间件的形式提供通信服务，并提供QoS（Quality of Service）策略，保障数据实时、高效、灵活的分发。

- **通信模型**：DDS采用以数据为中心的发布-订阅模型（Data-Centric Publish-Subscribe，DCPS），创建了一个“全局数据空间”的概念，所有独立的应用都可以访问。发布者将数据发布到特定主题，订阅者通过订阅主题接收感兴趣的数据，无需知道对方的存在。
- **核心特点**：
  - **解耦合**：发布者和订阅者在时间、空间和同步上完全解耦，系统更容易扩展。
  - **无中心架构**：具有**分布式**自动发现协议，应用位置无关，即插即用，不存在单点故障。
  - **灵活的QoS策略**：提供丰富的QoS策略，可满足不同应用对数据共享方式的需求，如可靠性、延迟保证等。
- **应用场景**：广泛应用于对实时性要求较高的领域，如分布式仿真、实时监控、工业自动化、国防等。
- 应用层：使用Fast DDS API 在分布式系统中实现通信的用户应用程序。
- DDS 层：DDS 通信中间件的实现。它允许部署一个或多个 DDS 域，其中同一域内的域参与者通过在域主题下发布/订阅来交换消息。
- RTPS 层：实现实时发布-订阅 (RTPS) 协议以支持与 DDS 应用程序的互操作性。该层充当传输层的抽象层。
- 传输层：快速 DDS可用于各种传输协议，例如不可靠传输协议 (UDP)、可靠传输协议 (TCP) 或共享内存传输协议 (SHM)。

DDS的相关标准包括核心协议(DDSI-RTPS，DDS-XTypes，DDS-Security，Interface Definition Language (IDL)…)，API(DDS C++ API，DDS Java API)，拓展协议(DDS-RPC，DDS-XML…)等13份协议集合。

在分布式系统中，DDS位于操作系统和应用程序之间，支持多种编程语言以及多种底层协议。这便是我们常说的跨平台。

SOME/IP通信技术简介:面向服务

[https://zhuanlan.zhihu.com/p/626401884]
[https://www.autosar.org/fileadmin/standards/R24-11/FO/AUTOSAR_FO_PRS_SOMEIPProtocol.pdf]
[https://some-ip.com/]
SOME/IP (Scalable service-Oriented MiddlewarE over IP) 是一种面向服务的车载通信协议，广泛应用于汽车电子系统中。以下是SOME/IP的主要通信模型：

1. 基本通信模式
   SOME/IP支持多种通信模式：
   1.1 请求/响应模式 (Request/Response)
   - 客户端发送请求，服务端返回响应
   - 典型的远程过程调用(RPC)模式
   - 用于需要确认的操作或数据获取

   1.2 事件通知 (Event/Notification)
   - 服务端主动向订阅的客户端发送数据
   - 基于发布/订阅模型
   - 用于状态变化通知或周期性数据更新

   1.3 字段通信 (Field Communication)
   - 结合了属性访问和事件通知
   - 支持getter、setter和notifier功能
2. 服务发现机制 (SOME/IP-SD)
   SOME/IP服务发现是协议的核心部分，包括：
   - **服务提供者**：通过OfferService消息宣告可用的服务
   - **服务消费者**：通过SubscribeEventGroup消息订阅所需服务
   - **服务发现报文**：用于服务的注册、订阅和维护
3. 通信流程
   典型SOME/IP通信流程：
   1. 服务启动并广播服务可用性(OfferService)
   2. 客户端发现所需服务
   3. 客户端订阅感兴趣的事件或字段
   4. 建立通信连接
   5. 进行数据交换(请求/响应或事件通知)
   6. 服务终止时发送StopOffer消息

2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11653)
2. esp32笔记[5]-基于I2S协议实现音频播放(9557)
3. 谈谈NAS的硬盘SATA背板(6740)
4. 三星平板的笔记模板(比例:16:10)(4329)
5. 电犀牛R66s软路由体验(4279)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[1]-blink闪灯(1)
4. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

4. 特点
    - 基于IP网络，支持TCP和UDP传输
    - 支持序列化和反序列化
    - 具备服务发现能力
    - 支持多种QoS级别
    - 设计考虑了汽车电子的实时性要求

SOME/IP的这些特性使其成为汽车电子系统中服务导向架构(SOA)的理想通信中间件。

RustDDS库简介

[https://github.com/Atostek/RustDDS]

- **概述**：RustDDS是一个纯Rust实现的Data Distribution Service（DDS）库，遵循OMG的DDS规范，使用RTPS协议进行网络通信，默认基于UDP/IP。
- **功能特点**：
    - **功能兼容**：虽然不完全遵循DDS对象API，但实现了功能兼容，同时遵循Rust的编程习惯和规范。
    - **与ROS2的接口**：提供了与ROS2通信的接口，比直接使用DDS更简单，推荐使用ros2-client。
    - **开发状态**：由Atostek Oy开发，目前实现足够与ROS2软件进行数据交换。
- **版本更新**：不断有新版本发布，如0.11.4版本改进了对SPDP（发现）公告的响应速度，更新了依赖版本等。

ROS 2分布式通信简介

- **通信模型**：ROS 2的通信模型基于DDS，包含参与者（Participant）、发布者（Publisher）、订阅者（Subscriber）、数据写入器（DataWriter）、数据读取器（DataReader）、话题（Topic）等关键概念。
- **特点**：
    - **去中心化**：不再依赖ROS1中的ROS Master，节点之间通过发现机制建立连接。
    - **灵活的通信方式**：支持多种通信方式，如进程内通信、节点间通信等。
    - **QoS策略**：继承了DDS的QoS策略，可根据应用需求灵活配置。
- **应用优势**：提高了系统的可靠性和扩展性，适用于构建复杂的分布式机器人系统。

log4rs简介

[https://github.com/estk/log4rs]
log4rs是一个功能强大且灵活的Rust日志记录库，具有以下特点：

- **灵活的配置方式**：支持通过TOML、YAML、JSON等多种格式进行配置，方便用户根据需求定制日志记录规则。
- **丰富的日志记录功能**：提供了多种日志输出目标（Appender），如控制台、文件、网络等，以及多种日志格式化方式。
- **高效的日志处理**：采用异步日志记录机制，减少对主线程的影响，提高程序性能。
- **灵活的日志级别控制**：支持设置不同的日志级别，如Error、Warn、Info、Debug、Trace等，方便用户根据需要过滤日志。

ros2-client和ros2-interfaces-humble简介

[https://github.com/Atostek/ros2-client]
[https://docs.rs/ros2-interfaces-humble/0.0.1/ros2_interfaces_humble/]
[https://docs.rs/crate/ros2-interfaces-humble/0.0.1/source/]

- **ros2-client**：是一个简化与ROS 2组件通信的Rust库，比直接使用DDS更简单，推荐用于与ROS 2组件通信。
- **ros2-interfaces-humble**：是ROS 2 Humble版本的接口定义，提供了与ROS 2通信所需的各种接口，方便开发者在Rust中实现与ROS 2的交互。

实现

1. 代码

```rust
#![allow(unused)]

//! 订阅/发布ros2格式消息

// 异步处理
use futures::StreamExt;
use tokio::time::{sleep, Duration};

// ros2消息通信协议(RustDDS)
use ros2_client::{ros2, rosout, ros2::policy, Context, MessageTypeName, Name, NodeName, NodeO

// 日志
use log::info;

// 使用Tokio的异步主函数宏
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // 初始化日志系统(从项目根目录开始))
    log4rs::init_file("./assets/rosdds_pub_sub.log4rs.yaml", Default::default())?;

    // 创建ROS2上下文和节点
    let context = Context::new()?;

    // 创建两个独立节点
    let mut pub_node = context.new_node(NodeName::new("/rustdds", "pub")?, NodeOptions::new()
    let mut sub_node = context.new_node(NodeName::new("/rustdds", "sub")?, NodeOptions::new()

    // 配置可靠传输的QoS策略
    let reliable_qos = ros2::QosPolicyBuilder::new()
        .history(policy::History::KeepLast { depth: 10 })
        .reliability(policy::Reliability::Reliable {
            max_blocking_time: ros2::Duration::from_millis(100),
        })
        .durability(policy::Durability::TransientLocal)
        .build();

    // 创建主题 (Topic)
    let chatter_topic = pub_node.create_topic(
        &Name::new("/", "chatter")?,
        MessageTypeName::new("std_msgs", "String"),
        &reliable_qos,
    )?;

    // 创建发布者 (Publisher)
    let chatter_publisher = pub_node.create_publisher::<String>(&chatter_topic, None)?;

    // 创建订阅者 (Subscription)
    let chatter_subscription = sub_node.create_subscription::<String>(&chatter_topic, Some(re

    // 启动并发任务: 发布消息
    let publisher_task = tokio::spawn(async move {
        let mut count = 0;
        let filler = "All work and no play makes ROS a dull boy. ".repeat(4);

        loop {
            count += 1;
            let message = format!("count={} {}", count, filler);
            println!("[发布端] 正在发送, 计数={} 长度={}", count, message.len());

            // 异步发布消息
            if let Err(e) = chatter_publisher.async_publish(message).await {
                eprintln!("发布失败: {:?}", e);
            }

            // 每2秒发送一次
            sleep(Duration::from_secs(2)).await;
        }
    });

    // 启动并发任务: 订阅消息
    let subscriber_task = tokio::spawn(async move {
       // loop{
            let mut subscription_stream = chatter_subscription
                .async_stream()
                .for_each(|result| async {
                    match result {
                        Ok((msg, _)) => println!("[订阅端] 收到消息: {}", msg),
                        Err(e) => eprintln!("接收错误: {:?}", e),
                    }
                });

            // 记录ROS日志
            rosout!(
                sub_node,
```

```
                ros2::LogLevel::Info,
                "节点初始化完成，开始监听消息"
            );

            subscription_stream.await
            // } // end loop
    });

    // 等待所有任务完成
    let (pub_res, sub_res) = tokio::join!(publisher_task, subscriber_task);
    pub_res?;
    sub_res?;

    Ok(())
}
```

2. 日志配置文件

   rosdds_pub_sub.log4rs.yaml

```
refresh_rate: 10 s

appenders:
  stdout:
    kind: console
  file_appender:
    kind: file
    path: "./result/rosdds_pub_sub.log4rs.log"
root:
  level: info
  appenders:
    - stdout
    - file_appender

loggers:
  rustdds:
    level: warn
  rustdds::dds::message_receiver:
    level: info
  rustdds::dds::fragment_assembler:
      level: warn
  rustdds::dds::with_key::datareader:
    level: info
  rustdds::dds::writer:
    level: info
  rustdds::dds::datasample_cache:
    level: debug
```

效果

```

����������������������������������������������������������������������������������������������
[订阅端] 收到消息: count=6 All work and no play makes ROS a dull boy. All work and no play makes
2025-04-05T20:49:45.545402+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:45.545564+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=7 长度=180
[订阅端] 收到消息: count=7 All work and no play makes ROS a dull boy. All work and no play makes
2025-04-05T20:49:47.548503+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:47.548737+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=8 长度=180
[订阅端] 收到消息: count=8 All work and no play makes ROS a dull boy. All work and no play makes
2025-04-05T20:49:49.551369+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:49.551626+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=9 长度=180
[订阅端] 收到消息: count=9 All work and no play makes ROS a dull boy. All work and no play makes
2025-04-05T20:49:51.553391+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:51.553671+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=10 长度=181
[订阅端] 收到消息: count=10 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:49:53.555137+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:53.555304+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=11 长度=181
[订阅端] 收到消息: count=11 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:49:55.555624+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:55.555736+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=12 长度=181
[订阅端] 收到消息: count=12 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:49:57.557297+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:57.557415+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=13 长度=181
[订阅端] 收到消息: count=13 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:49:59.559929+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:49:59.560056+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=14 长度=181
[订阅端] 收到消息: count=14 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:01.562633+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:01.562981+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=15 长度=181
[订阅端] 收到消息: count=15 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:03.564330+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:03.564504+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=16 长度=181
[订阅端] 收到消息: count=16 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:05.567899+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:05.568263+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送, 计数=17 长度=181
[订阅端] 收到消息: count=17 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:07.568514+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
```

```
2025-04-05T20:50:07.568647+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=18 长度=181
[订阅端] 收到消息：count=18 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:09.571365+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:09.571483+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=19 长度=181
[订阅端] 收到消息：count=19 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:11.575427+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:11.575812+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=20 长度=181
[订阅端] 收到消息：count=20 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:13.577102+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:13.577231+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=21 长度=181
[订阅端] 收到消息：count=21 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:15.579240+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:15.579441+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=22 长度=181
[订阅端] 收到消息：count=22 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:17.582547+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:17.582984+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=23 长度=181
[订阅端] 收到消息：count=23 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:19.584288+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:19.584446+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=24 长度=181
[订阅端] 收到消息：count=24 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:21.586916+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:21.586994+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=25 长度=181
[订阅端] 收到消息：count=25 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:23.587328+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:23.587424+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=26 长度=181
[订阅端] 收到消息：count=26 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:25.589781+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:25.589864+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=27 长度=181
[订阅端] 收到消息：count=27 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:27.591264+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:27.591332+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=28 长度=181
[订阅端] 收到消息：count=28 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:29.594969+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:29.595315+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=29 长度=181
[订阅端] 收到消息：count=29 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:31.598100+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:31.598531+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=30 长度=181
[订阅端] 收到消息：count=30 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:33.600431+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:33.600900+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=31 长度=181
[订阅端] 收到消息：count=31 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:35.602362+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:35.602717+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=32 长度=181
[订阅端] 收到消息：count=32 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:37.603776+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:37.603859+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=33 长度=181
[订阅端] 收到消息：count=33 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:39.605136+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:39.605217+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=34 长度=181
[订阅端] 收到消息：count=34 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:41.607636+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:41.607744+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=35 长度=181
[订阅端] 收到消息：count=35 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:43.610177+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:43.610281+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=36 长度=181
[订阅端] 收到消息：count=36 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:45.612920+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:45.613031+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=37 长度=181
[订阅端] 收到消息：count=37 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:47.614755+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:47.614948+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
[发布端] 正在发送，计数=38 长度=181
[订阅端] 收到消息：count=38 All work and no play makes ROS a dull boy. All work and no play make
2025-04-05T20:50:49.616919+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
2025-04-05T20:50:49.617171+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
```

分析

```
好的，用户给出了一段Rust代码和对应的日志输出，希望我分析日志并解释其中的内容。首先，我需要明确用户的需求：他们
我先快速浏览了一下日志，发现它主要包含三部分：发布端的日志、订阅端的日志，以及一些警告信息。发布端和订阅端的日志
我的第一反应是，这些警告信息可能与网络配置或DDS库的行为有关。日志中提到的“Invalid argument”错误通常与网络地
接下来，我仔细检查了代码，发现它使用了RustDDS库来创建节点和主题，并且没有显式指定网络地址或端口。这意味着DDS库
在查看文档时，我发现RustDDS默认会尝试多种地址和端口组合进行网络发现，如果某些地址无效，就会产生类似的警告。这
为了确保我的分析是全面的，我还需要考虑其他可能性，比如是否是日志级别设置的问题。我检查了日志配置文件，发现警告级
最后，我需要决定如何向用户解释这些分析结果。我决定按照以下逻辑组织回复：
先描述日志的总体结构，让用户对日志内容有一个清晰的了解。
然后解释发布端和订阅端的日志，强调它们的正常性。
接着分析警告信息，解释它们的来源和原因，并说明这些警告是预期行为，不需要修复。
最后总结整个分析过程，确保用户对日志输出有一个全面的理解。
我准备好回复用户了。
```

回答：
从代码和日志输出来看，这段程序是一个基于ROS2的发布/订阅（Publisher/Subscriber）示例，使用了Rust语言和RustDDS库。以下是对日志输出的详细分析：

1. 日志结构分析

日志输出主要分为三部分：

1. **发布端日志**：显示消息发送的状态。
2. **订阅端日志**：显示接收到的消息内容。
3. **警告日志**：由RustDDS库生成，与网络通信相关。

2. 发布端日志

```
[发布端] 正在发送，计数=6 长度=180
```

- **计数**：表示发送的消息序号（从1开始递增）。
- **长度**：表示消息内容的字符长度。
- **说明**：发布端每2秒发送一次消息，消息内容是动态生成的，包含计数和固定字符串。

3. 订阅端日志

```
[订阅端] 收到消息: count=6 All work and no play makes ROS a dull boy. ...
```

- 订阅端成功接收到发布端发送的消息，并打印了完整的内容。
- 消息内容与发布端发送的内容一致，说明通信是正常的。

4. 警告日志

```
2025-04-05T20:49:45.545402+08:00 WARN rustdds::network::udp_sender - send_to_udp_socket - sen
```

- **来源**：警告来自RustDDS库的 `rustdds::network::udp_sender` 模块。
- **内容**：在尝试通过UDP发送数据时，出现了 `Invalid argument` 错误。
- **原因**：
  - 这是由于网络地址或端口配置问题导致的。
  - 可能是发送端尝试向无效的IPv6地址或端口发送数据。
  - 这种警告通常是网络发现（Discovery）过程中的一部分，属于正常现象，不会影响通信。

5. 日志总结

- **通信正常**：发布端和订阅端之间的消息传递是成功的，订阅端能够正确接收到消息。
- **警告原因**：警告是由于RustDDS在发现网络邻居时尝试了无效的地址或端口，这是预期行为，不影响通信。
- **优化建议**：
  - 如果需要减少警告，可以调整网络配置或日志级别（如将 `rustdds::network::udp_sender` 的日志级别设置为 `error`）。

6. 代码优化建议

1. **日志级别调整**：
   在 `rosdds_pub_sub.log4rs.yaml` 中，将 `rustdds::network::udp_sender` 的日志级别从 `warn` 改为 `error`，以减少无关警告：

   ```
   loggers:
     rustdds::network::udp_sender:
       level: error
   ```

2. **网络配置检查**：
   确保运行环境的网络配置（如IPv6地址）是有效的，避免无效地址导致的警告。

3. **错误处理改进**：
   在代码中增加对发布失败的处理逻辑，例如记录失败次数或重试机制：

```
if let Err(e) = chatter_publisher.async_publish(message).await {
    eprintln!("发布失败: {:?}", e);
    // 可以在这里添加重试逻辑
}
```

7. 总结

- **通信正常**：发布端和订阅端的通信是成功的。
- **警告无害**：警告是RustDDS库在发现网络邻居时的正常行为，可以忽略。
- **优化方向**：调整日志级别或网络配置以减少无关警告。

备注

本文专用于AI搜索和建设AI知识库.

分类: 机器人 , 折腾笔记

标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 折腾笔记[18]-使用rust读取rosbag数据

posted @ 2025-04-05 21:50 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· 大模型 Token 究竟是啥：图解大模型Token
· 35岁程序员的中年求职记：四次碰壁后的深度反思
· 继承的思维：从思维模式到架构设计的深度解析
· 如何在 .NET 中 使用 ANTLR4
· 后端思维之高并发处理方案

阅读排行：
· BotSharp + MCP 三步实现智能体开发
· BotSharp 5.0 MCP：迈向更开放的AI Agent框架
· 5. RabbitMQ 消息队列中 Exchanges(交换机) 的详细说明
· 设计模式脉络
· 【ESP32】两种模拟 USB 鼠标的方法

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes